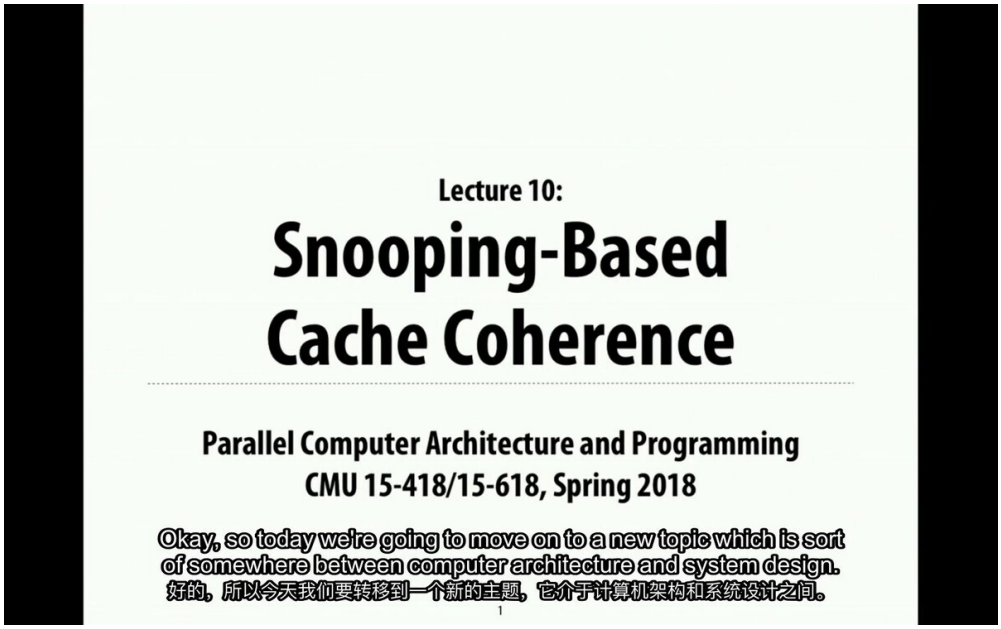
课程笔记

基于监听的缓存一致性：MSI 协议与伪共享

CMU 15-418/618 并行计算机架构与编程 · Lecture 11（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 20 分 47 秒
视频链接：`https://www.bilibili.com/video/BV1HN411z7gD/`

目录

1 缓存一致性：一个被缓存制造出来的错觉 **3**

1.1 先把缓存的基本行为过一遍 . . . 3
1.2 回写与写直达 . . . 3
1.3 写缺失时的完整步骤 . . . 4
1.4 本章小结 . . . 4

2 一致性问题到底出在哪 **5**

2.1 一个最小反例 . . . 5
2.2 为什么必须由硬件解决 . . . 6
2.3 一致性动作发生在哪一层 . . . 7
2.4 单核也不简单 . . . 7
2.5 本章小结 . . . 8

3 一致性的精确定义 **8**

3.1 为什么不能直接要求顺序一致性 . . . 8
3.2 三条不变量 . . . 10
3.3 违背写串行化会怎样 . . . 11
3.4 本章小结 . . . 11

4 snooping 的基本思路 **12**

4.1 方案选择 . . . 12
4.2 snooping 的机制 . . . 13
4.3 协议讨论的三个前提 . . . 13
4.4 本章小结 . . . 14

5 入门协议：写直达加失效 **14**

5.1 假设与做法 . . . 14
5.2 两状态状态图 . . . 15
5.3 为什么不够 . . . 15
5.4 本章小结 . . . 16

6 MSI 协议 **16**

6.1 回写缓存带来的新问题 . . . 16
6.2 三个状态 . . . 17
6.3 总线上的三类事务 . . . 17
6.4 状态转移的关键行为 . . . 18
6.5 一个执行示例 . . . 18
6.6 本章小结 . . . 20
6.7 MSI 是否满足一致性 . . . 20

7 MSI 的代价与 MESI 的改进 **21**

7.1 来回争抢 . . . 21
7.2 伪共享：程序员的视角 . . . 22

7.3 不同系统的取舍 22

8 总结与延伸 **22**

1　缓存一致性：一个被缓存制造出来的错觉

这一讲进入一个新主题，讲者说它介于计算机体系结构与系统设计之间。当你需要写出能在大型共享内存机器上运行的代码时，它变得相当重要。

问题本身是：在一个有多份缓存、但内存是共享的系统里，如何让这些缓存彼此保持一致。

讲者指出这对程序员有实际影响：你的目标机器如何实现一致性，会相当深刻地影响性能。

1.1　先把缓存的基本行为过一遍

讲者用一个具体例子回顾缓存设计。假设代码里执行 `volatile int x = 1`。在 C 或 C++ 中，`volatile` 意味着写就是真的写进内存，而不只是落在寄存器里。

假设这个 4 字节变量位于地址 `12345604`，缓存行是 64 字节（相当典型）。那么看低 6 位就能确定行内偏移——找到偏移是 4。于是从这一行的第 4 个字节开始写入，小端序下看起来是 1 0 0 0。

行内偏移由地址的低位直接决定：

$$\text{offset} = A \bmod 2^{\log_2 L}$$

- A：变量在内存中的地址，本例为 `12345604`；
- L：缓存行的字节数，本例为 64，故指数为 6；
- offset：该字节在缓存行内的起始位置，本例为 4。

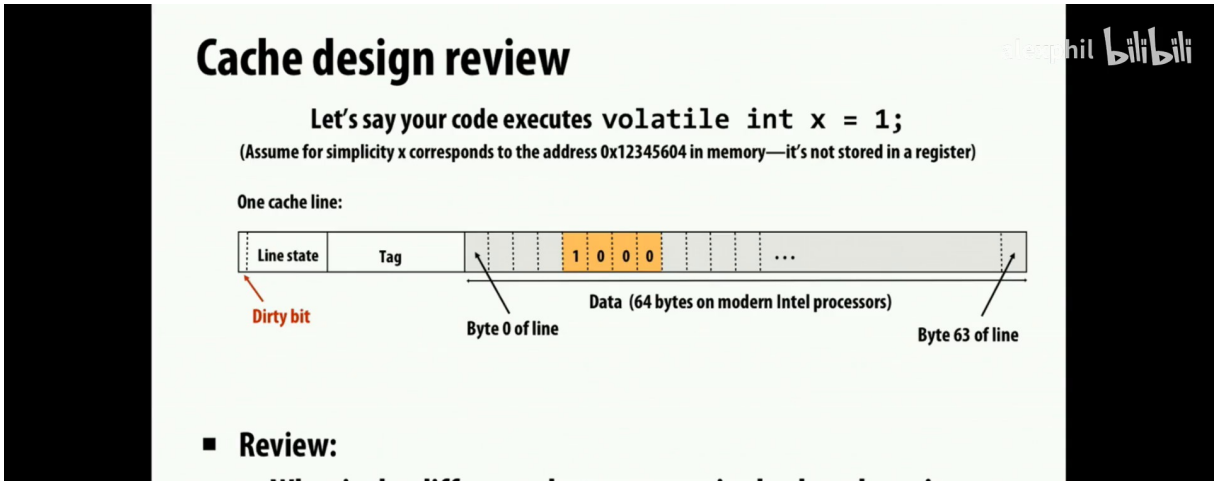


图 1: 缓存设计回顾：volatile 写与缓存行内的偏移定位[1]

1.2　回写与写直达

讲者接着回顾两种写策略，这个区分在后面会直接决定协议的复杂度。

回写是指写只落在缓存里，等到某一刻必须覆盖这一行时（容量淘汰，或进程结束），才强制把它写回内存。

写直达是指每一次写都一直穿透到内存。

[1] 视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

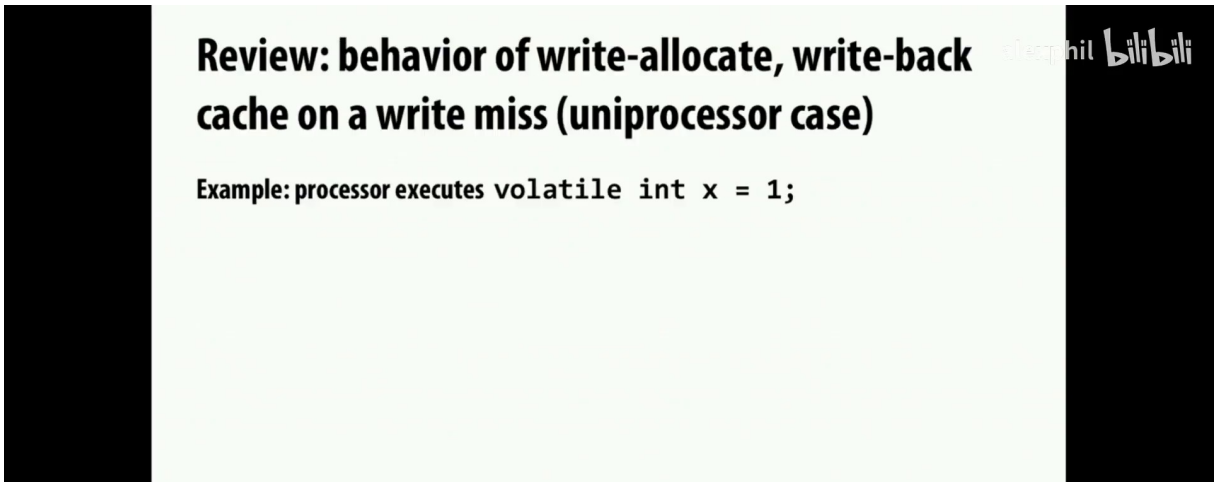


图 2: 回顾：写分配、回写缓存在写缺失时的行为[2]

写直达更容易实现，但性能明显更低。它能站住脚的原因是：大多数数据被读的次数远多于被写的次数。

写策略与一致性协议是一对绑定关系

写直达让内存始终是最新的，因此一致性协议只要处理”让别人丢弃副本”这一件事。回写让缓存可能比内存更新，协议就必须额外处理”谁持有最新值、何时交出”的问题。这就是为什么本讲先讨论写直达的两状态协议，再推进到 MSI 三状态协议。

1.3 写缺失时的完整步骤

讲者把一次写缺失的处理拆开：如果之前没有读过这个地址、缓存里没有这一行，就要先按放置策略决定它该放哪。放置策略通常是组相联映射限制可能性，再用类似 LRU 的方式在一组里挑牺牲者。

然后分配这个块，把它的当前内容读进缓存，再更新我们关心的那些字节，并把这一行标记为脏。

这里的”脏”有明确含义：缓存里的值与内存里的值不一致。

脏位是”内存已过时”的标记

脏位不是”这个数据很重要”，而是”这份缓存副本比内存新”。它决定了淘汰时必须先写回内存，也决定了多处理器下必须由持有脏副本的缓存负责交出数据。理解这一点，才能理解后面 MSI 中 M 状态为何要承担额外责任。

1.4 本章小结

单处理器上的缓存机制本身已包含若干细节：行大小与偏移决定一次写落在何处，写直达与回写决定内存是否会过时，脏位记录两者的差异。这些都是”单机自洽”的。

一旦有多个处理器各自缓存同一批数据，这套机制就会给出彼此矛盾的视图。下一章把这个矛盾具体化。

[2]视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

2　一致性问题到底出在哪

2.1　一个最小反例

讲者把问题摆出来：在一个共享内存多处理器上，通常我们希望，如果某个处理器写了位置 x，另一个处理器读同一位置时，应当看到那次写。否则就谈不上”共享”。

这个要求听起来天经地义，但缓存让它变得困难，尤其是回写缓存——某个处理器持有的副本可能比主存里的更旧或更新。

他给出的最小场景是这样的：处理器 1 读到 x 的初值 0，把它带进缓存；处理器 2 也读 x，同样从内存拿到 0。到这一步都是对的。

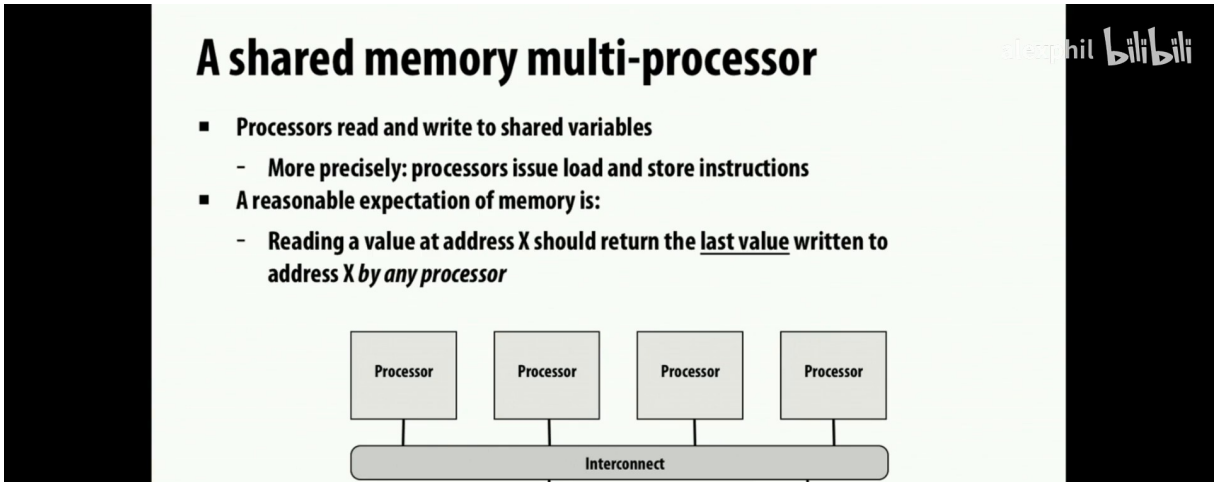


图 3: 共享内存多处理器：处理器读写共享变量[3]

然后处理器 1 往 x 写入新值。即便用写直达，处理器 2 仍然以为它缓存里有 x，而它持有的值仍是 0。除非我们做点什么，否则它会一直认为自己看到的是 0。

若用的是回写缓存，情形更糟：处理器 3 来读 x 时，会从内存读到仍然是 0 的旧值。

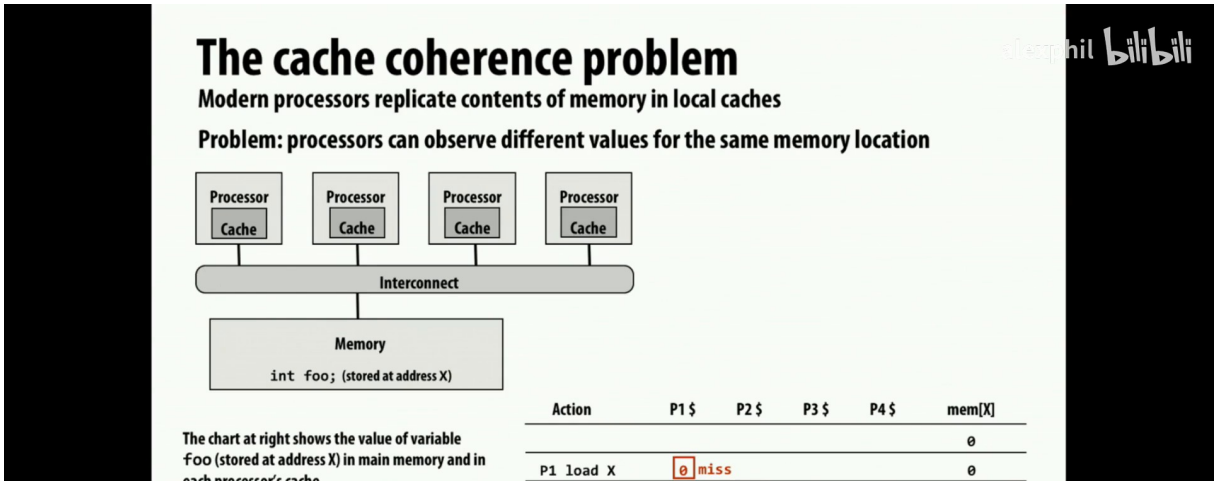


图 4: 缓存一致性问题：现代处理器在本地缓存中复制可写数据[4]

[3]视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

[4]视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

2.2 为什么必须由硬件解决

讲者强调这件事必须在硬件中处理：缓存的运作在纳秒量级，靠软件手段管理不可能有可接受的性能。

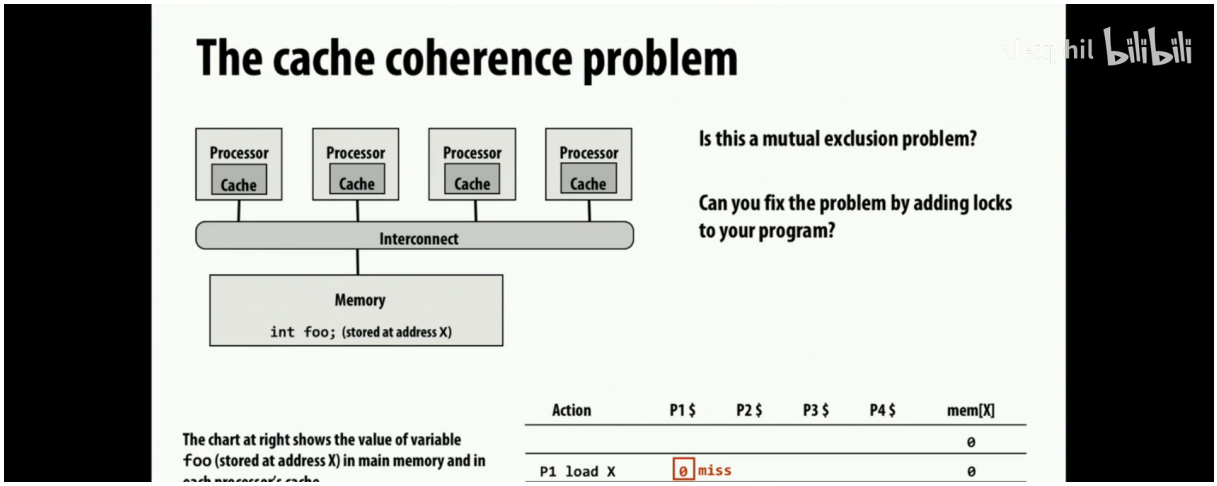


图 5: 这算是互斥问题吗：一致性问题的另一种表述[5]

他在这里做了一个概念澄清：我们想要的是——每当读取位置 x，读到的应当是该地址被任何处理器最近写入的值。

这个目标其实在维护一个错觉。缓存的全部意义是把内存层次藏起来，让程序员以为存在一份单一的内存。但物理上那只是一堆分层的存储。讲者的原话是：这是我们作为缓存设计者的职责，把这个错觉修好。

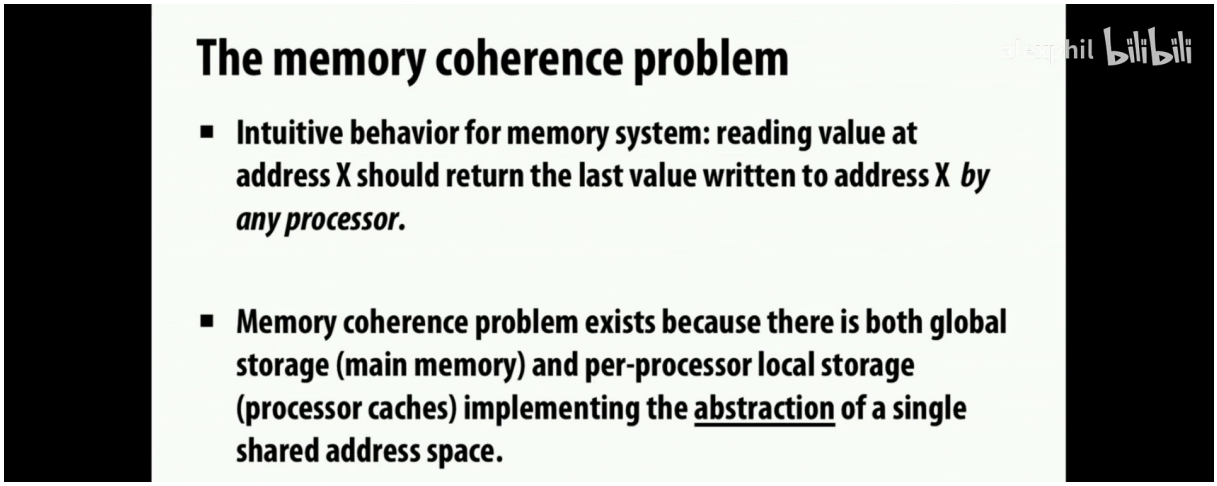


图 6: 内存一致性问题：对内存系统行为的直觉预期[6]

[5] 视频画面时间区间：00:10:00–00:10:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

不依赖一致性保证时，正确性由软件承担

在不提供一致性的平台上，跨线程共享可写数据会得到不确定结果。若确实需要共享，必须使用平台明确支持的同步原语（原子操作、栅栏），而不能依赖”读完就能看到别人写的值”这一直觉。这是移植代码时最常见的隐性 bug 来源。

他也指出，市面上确实存在不保证一致性的系统。但如果我们希望用户把内存当成抽象的单一对象、让缓存与否不改变程序算出的结果，那就必须处理它。

2.3 一致性动作发生在哪一层

讲者展示了常见的缓存层次：从处理器向外，有两级每核私有的缓存，然后是一个大得多的、所有核心共享的 L3，再往外是主内存。

关键在于：处理器之间的一致性动作发生在 L2 与 L3 之间，那里正是可以插入硬件维持一致性的位置。

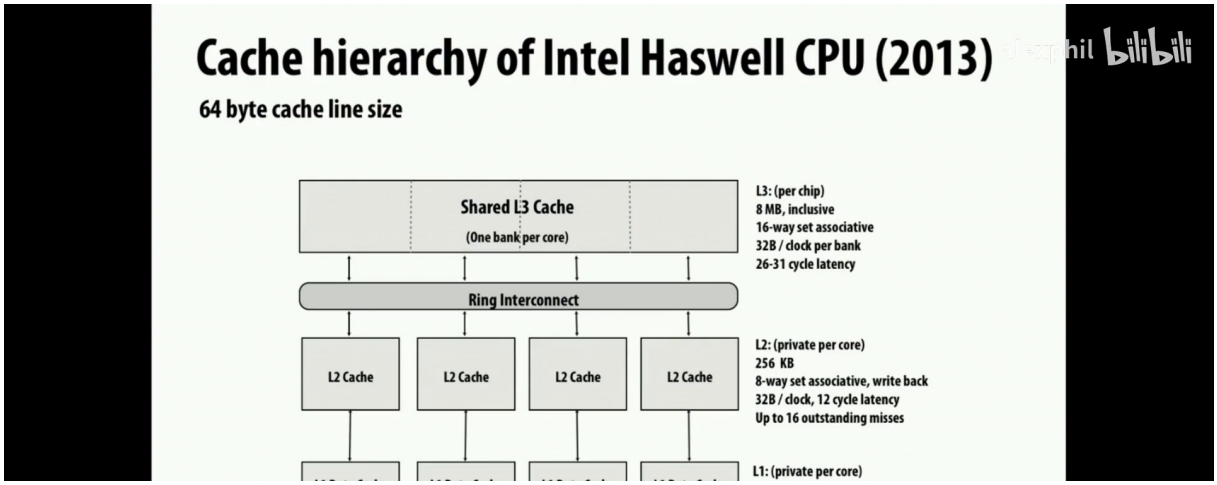


图 7: Intel Haswell（2013）的缓存层次：L1/L2 私有，L3 共享[7]

他还提到一个实现细节：物理上 L3 被切成若干扇区或分块，每一块与某个核心物理关联，但硬件把它当作单一的一致单元来管理。因此除了轻微的性能影响，程序员看不到这个划分。

2.4 单核也不简单

讲者顺带指出：即使在单个处理器内部，保持一致也不平凡。由于流水线设计，处理器维护着一个待定写操作的缓冲队列，类似 FIFO。每次加载都必须扫描所有待定的写，找出最近的那条。

另外，DMA 也是麻烦来源——IO 通常通过它工作：处理器填好内存缓冲，然后通知控制器取走；读磁盘时则是给控制器一个内存位置，让它做完后发中断通知。于是即使在单 CPU 上，也有两个主体在读写同一批内存。

[7]视频画面时间区间：00:12:00–00:12:15。若为裁剪图，时间区间仍对应原视频画面。

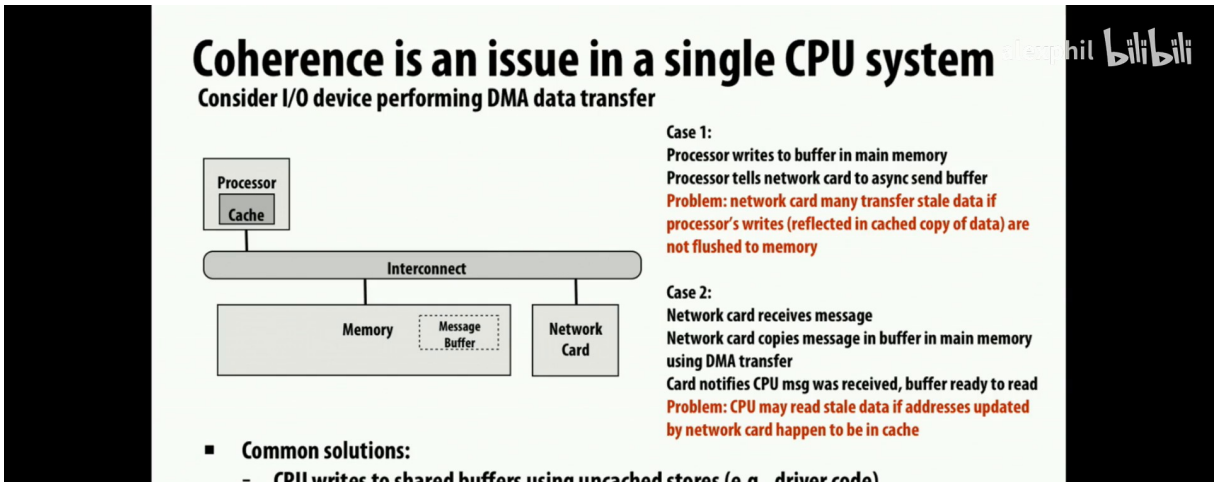


图 8: 单 CPU 系统中一致性同样是问题：DMA 让设备成为第二个主体[8]

讲者说这些单处理器的问题我们今天先假定已解决，专心对付多处理器的问题。

2.5 本章小结

一致性问题的最小形态是：两个处理器缓存了同一地址，一个写入后，另一个仍持有旧副本。回写缓存让情况更糟，因为内存本身也可能是过时的。这类问题必须在硬件里解决，因为时间尺度在纳秒级。在真实机器上，一致性逻辑插在私有缓存与共享缓存之间。

要谈”解决”，先要说清”保证什么”。这需要一个精确定义。

3 一致性的精确定义

3.1 为什么不能直接要求顺序一致性

最直觉的定义是：看起来像是所有内存事务按某个全局顺序发生。

讲者说这不是一个好的操作性定义。除非真的放一个单一权威机构来决定事件顺序，否则顺序一致性并不是可行的实现策略。

[8]视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

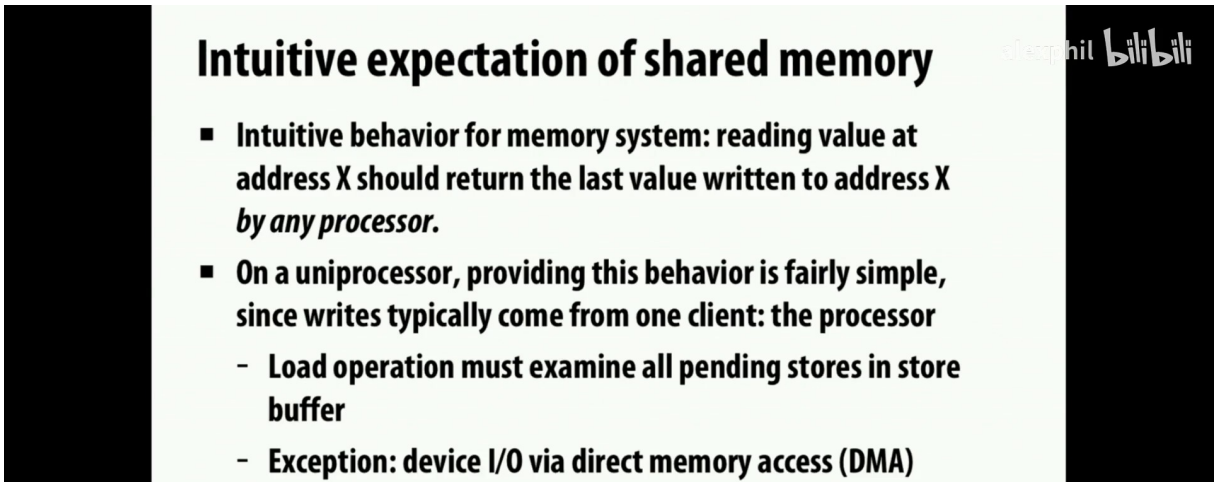


图 9: 对共享内存的直觉预期：读到最近的写[9]

于是他改为按”实际能实现什么”来描述。

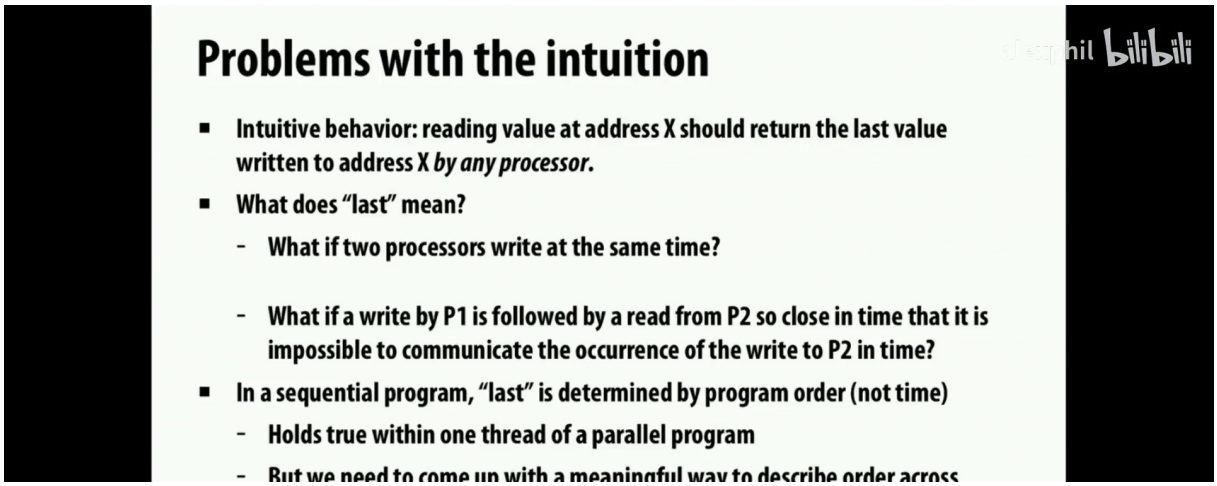


图 10: 直觉的问题：读写交互的实际行为需要更精确的定义[10]

[9]视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

[10]视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

3.2 三条不变量

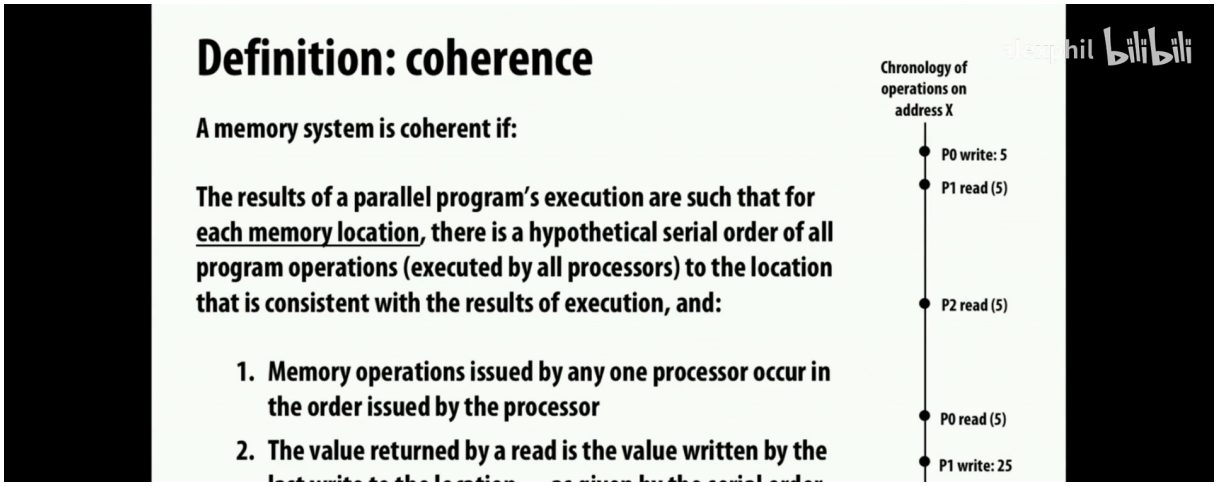


图 11: 一致性的定义：每个处理器对同一地址的操作必须符合程序顺序[11]

第一条是本地视角：若处理器 p 读取地址 x，它应当读到 p 自己最近写入的值，除非期间有其他处理器对该地址写过。这相当于单处理器上的串行化。

形式化地写成：

$$\text{read}_p(x) \Rightarrow \text{value} = \text{last_write}_p(x) \quad \text{若无他人介入}$$

- $\text{read}_p(x)$：处理器 p 对地址 x 的读；
- $\text{last_write}_p(x)$：p 自己最近一次对 x 的写；
- 条件”若无他人介入”排除期间其他处理器写过 x 的情形。

第二条是最终可见：若处理器 1 对 x 写入，经过足够时间后处理器 2 读 x 就能读到那个值。讲者承认”足够时间”这个说法有点含糊，但这就是最终一致性或最终可见的意思。

写串行化是全局共识而非局部顺序

第一条只约束单个处理器自己的读写顺序，第三条要求不同处理器之间对同一地址的写顺序达成共识。前者靠本地信息即可维护，后者必须借助某种全局串行化机制（往往是互连）。这正是协议实现中最昂贵、也最容易出错的部分。

第三条是写串行化：如果对同一地址有多次写，所有可能观察到这些写的处理器，必须以相同的顺序看到它们。

三条不变量合起来才等于可线性化

第一条保证每个处理器自己的读写顺序不被颠倒；第二条保证写最终会被看到；第三条保证所有观察者对同一地址的写入顺序有共识。少了任何一条，整个系统就不存在一个能解释全部观测的全局顺序。

[11] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

3.3 违背写串行化会怎样

讲者用构造性例子说明第三条为什么必要。

设处理器 1 向 x 写入值 A，处理器 2 向 x 写入值 B。处理器 3 与处理器 4 各自做一串读。

若处理器 3 读到的是先 A 后 B，而处理器 4 读到的是先 B 后 A，那就没有任何办法把这个读写序列串行化并得到一致的结果。这种情况违背了顺序一致性这个全局目标，因此是不可接受的方案。

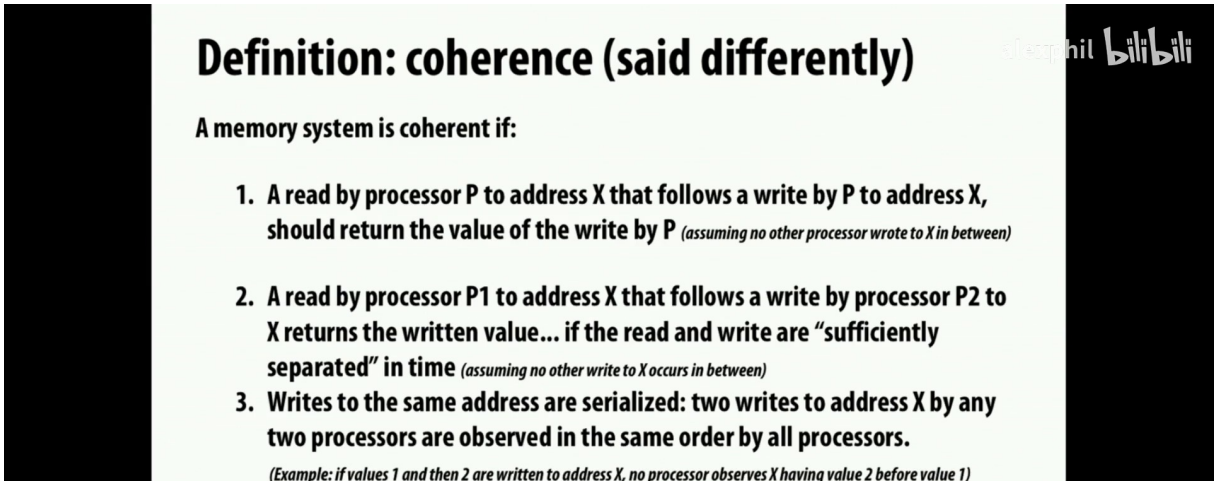


图 12: 换一种说法的一致性定义：三条要求并列表述[12]

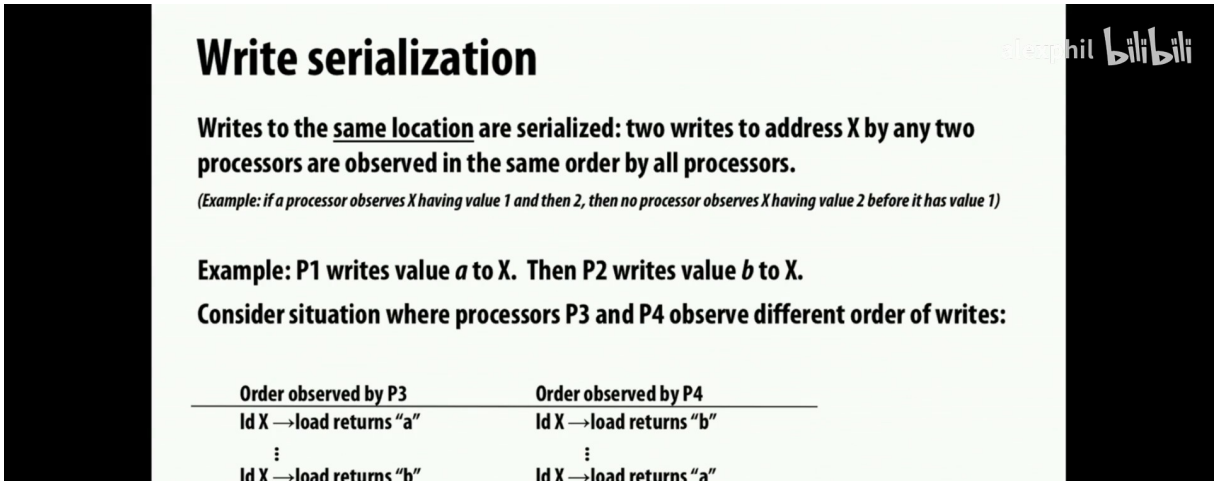


图 13: 写串行化：同一地址的两次写必须让所有观察者看到相同顺序[13]

讲者说可以证明：只要遵守这三条，就能保证所需的可串行化，但这个证明超出本课范围。

3.4 本章小结

一致性的可行定义由三条不变量组成：单处理器的读写顺序、写的最终可见性、同一地址写的全局顺序一致。第三条的必要性可以用” 两个读者看到相反顺序” 的构造证明。这个定义是可以落地的，而” 存在一个全局全序” 的表述则难以直接实现。

[12] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

[13] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

有了定义，就可以讨论实现方案。

4 snooping 的基本思路

4.1 方案选择

讲者先列出实现一致性的几条路。软件方案（例如操作系统用页错误机制传播更新）太慢——那是毫秒量级，而这里需要纳秒量级。

剩下的主流做法是：让共享缓存承担中心角色。既然一个缓存是共享的，它就是一个能监听内存事务的全局实体，可以据此决定如何搬运数据以保持一致。

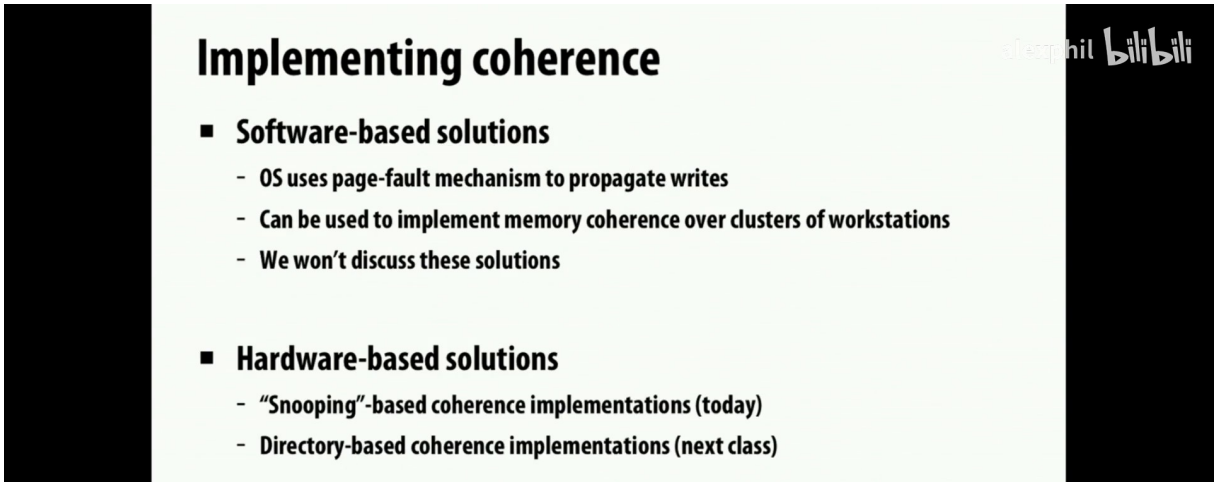


图 14: 实现一致性的方案选择：软件方案太慢[14]

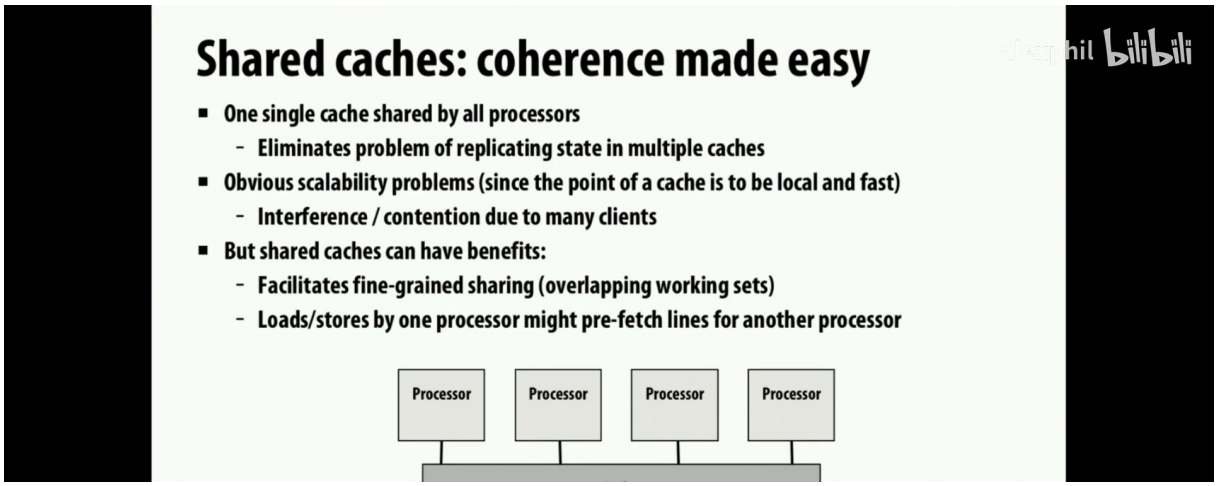


图 15: 共享缓存让一致性变得容易：单一缓存消除了私有副本[15]

[14] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:25:00–00:25:15。若为裁剪图，时间区间仍对应原视频画面。

4.2 snooping 的机制

今天讲的是最简单、也最标准的做法：基于监听（snooping）的缓存一致性协议。讲者预告下一讲会讨论更具扩展性的方案——目录式。

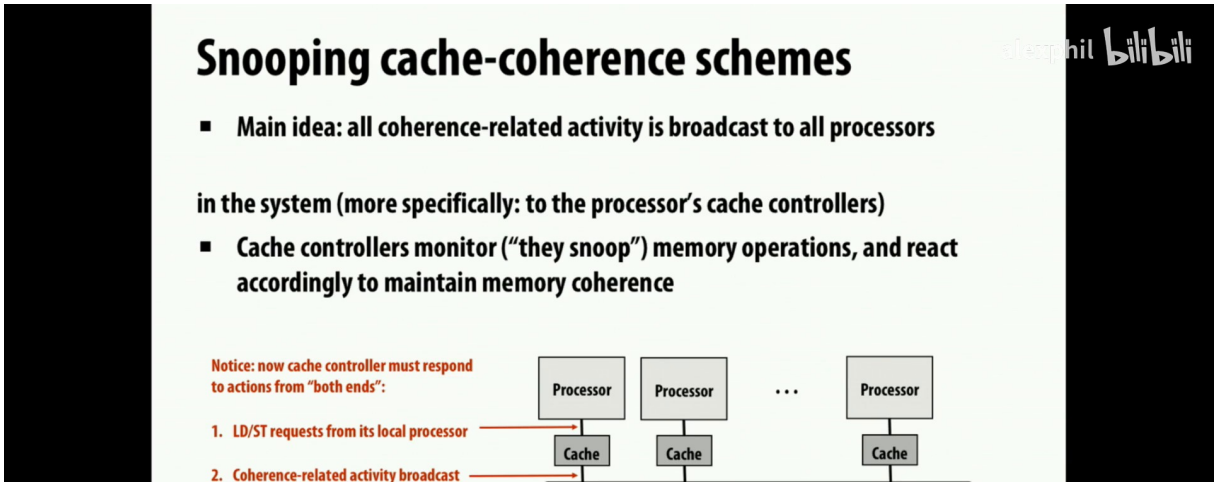


图 16: snooping 方案：所有与一致性相关的活动广播到共享互连[16]

核心是利用缓存的集中性。这个共享缓存提供了一个全局实体，可以监听内存事务、决定如何移动数据，让它看起来是一致的。

具体做法是给缓存控制器增加逻辑，使它们不只响应自己处理器的请求，也响应来自互连的请求。互连上会有消息飞来飞去，是各缓存之间在互相通告" 我关心的那些行现在是什么状态"。

代价随之而来：互连上会有流量，不只是缓存到内存的信息，还有缓存到缓存的通信。讲者指出这会限制可扩展性，但方案相当直接，也是当今大多数多核处理器采用的做法。

snooping 的代价随核心数增长

广播机制意味着每个一致性事件都要通知所有缓存。核心数增加时，互连流量按参与者数量增长，而互连带宽增长有限。因此 snooping 的可扩展性有天然上限，这也正是后续要引入目录式协议的原因。在核数很多时，即使正确性无问题，性能也会被一致性流量拖住。

4.3 协议讨论的三个前提

讲者交代了讨论协议时要注意的几点。

第一，每个缓存控制器负责管理自己本地缓存的状态。它们之间通过消息互相告知该做什么，同时也响应与自己缓存绑定的处理器的加载与存储请求。

第二，和任何协议一样，前提是所有缓存都遵守同一套规则。如果有一个缓存行为异常（对抗性的或设计错误的），它就能破坏整个系统。

第三，状态是以缓存行为单位记录的。为简单起见，暂时只考虑单字缓存行，因此不必处理" 为了改一个字节要先读入整行" 的问题。

[16] 视频画面时间区间：00:25:30–00:25:45。若为裁剪图，时间区间仍对应原视频画面。

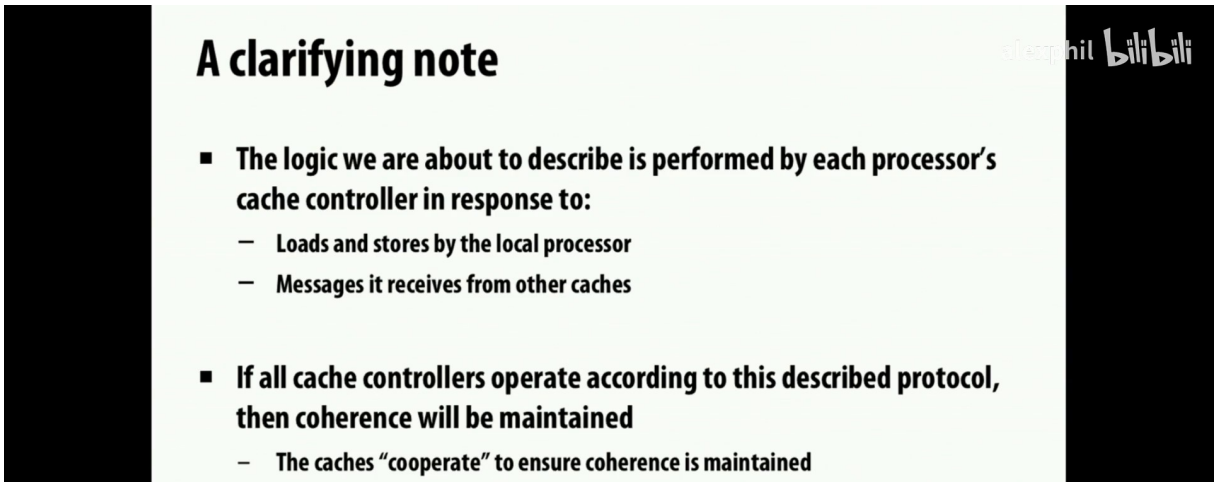


图 17: 一处澄清：描述的逻辑由每个处理器的缓存控制器执行[17]

4.4 本章小结

一致性必须由硬件实现，主流方案是把共享缓存当作监听中心，各缓存控制器在互连上广播事务。代价是互连流量随参与者增长，限制了可扩展性。协议的讨论以缓存行为单位、并假设所有控制器遵守同一套规则。

下面从最简单的协议开始。

5 入门协议：写直达加失效

5.1 假设与做法

讲者先设定最简化的情形：写直达缓存，单字缓存行。

于是每次处理器写，缓存就向内存写直达，并且在互连上发出消息说" 我在写地址 X"。任何其他持有该地址副本的处理器，在这个简化版本里就应当作废自己的条目。

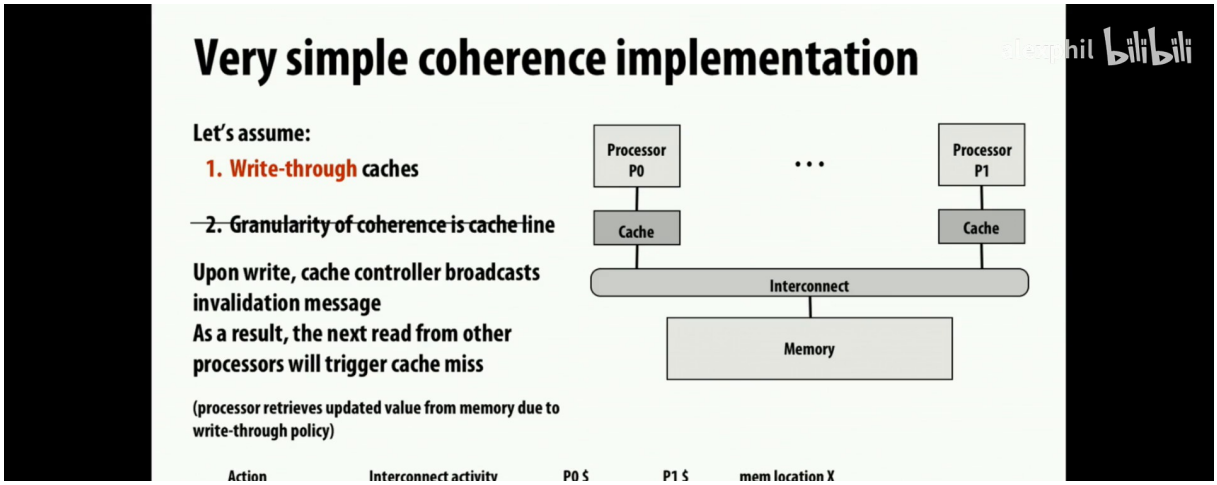


图 18: 非常简单的实现：写直达加失效[18]

[17] 视频画面时间区间：00:30:30–00:30:45。若为裁剪图，时间区间仍对应原视频画面。

这是一个基于失效（invalidation）的协议：通过作废可能变得陈旧的副本，来维持一致。讲者提到还有另一类基于更新（update）的协议，会在稍后瞥一眼。

失效与更新是处理陈旧副本的两条路

失效协议广播”这行变脏了，你们作废”，之后谁要用谁再取一次。更新协议广播”这行的新值是……”，让其他副本直接跟上。前者在写频繁而读稀少时更省带宽，后者在多个读者反复读同一行时更省延迟。商用多核几乎都选失效。

5.2 两状态状态图

这样一个协议可以用极简的状态图描述：每个缓存行只有两个状态，有效或无效。

因为是写直达，不存在”脏行”这种东西。

讲者说明了状态图的约定：实线表示由处理器动作引发的迁移，虚线表示由其他缓存控制器发来的消息引发的迁移。每条弧上左边写触发事件，右边写控制器应当执行的动作。

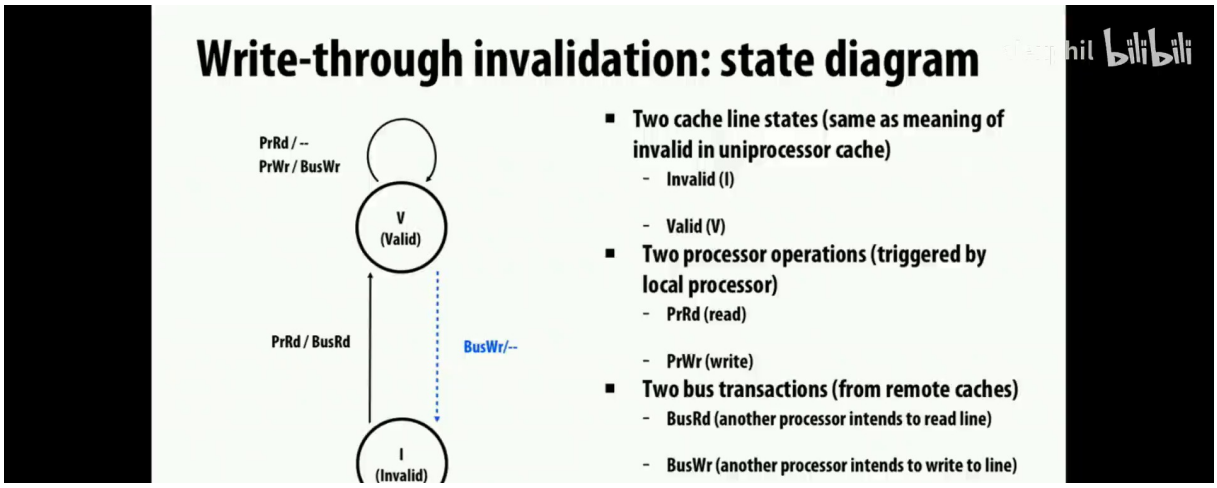


图 19: 写直达加失效的状态图：只有有效与无效两个状态[19]

典型路径是这样的：从一个无效行开始，读取它，把它带入缓存并标记有效，然后读值。此后若只是反复读，它可以一直保持有效，无需产生任何额外流量。

5.3 为什么不够

讲者随即指出这个策略效率不高：每次写操作都要走到内存，带来很高的总线流量。

在简单场景下这还能接受，但它把一致性的成本与写操作的频率直接绑定了——写越多，总线越挤。这就促使向回写缓存下的协议演进。

[18] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[19] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

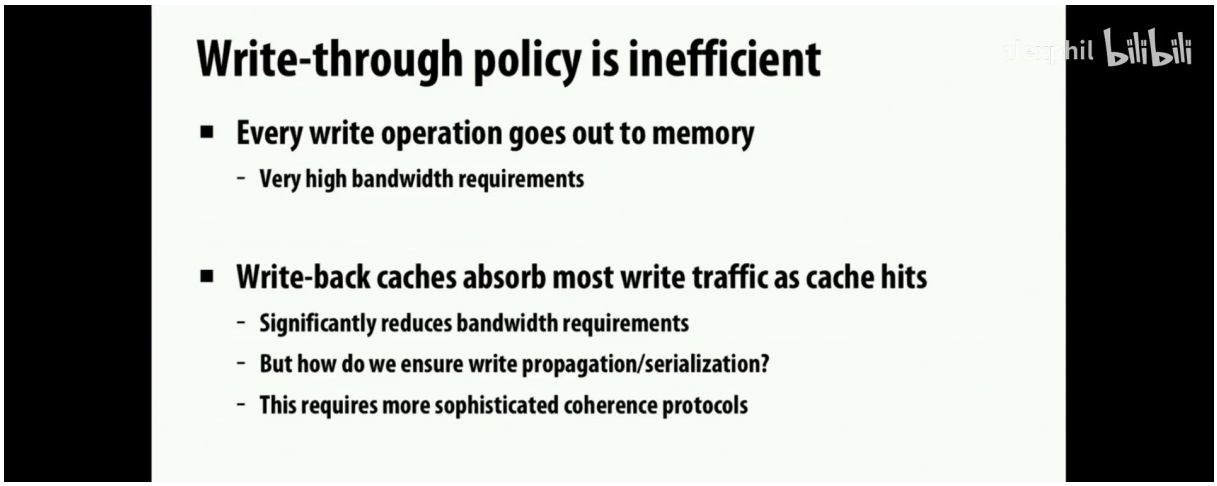


图 20: 写直达策略效率不佳：每次写都要占用总线[20]

5.4 本章小结

写直达加失效的两状态协议是把一致性引入缓存的最小可行方案：写时广播、他人作废。它逻辑清楚，但因为每次写都要上总线而不够高效，也无法利用回写带来的本地写优势。

要在回写缓存上做一致性，就必须面对”缓存可能持有比内存更新的副本”这个问题。

6 MSI 协议

6.1 回写缓存带来的新问题

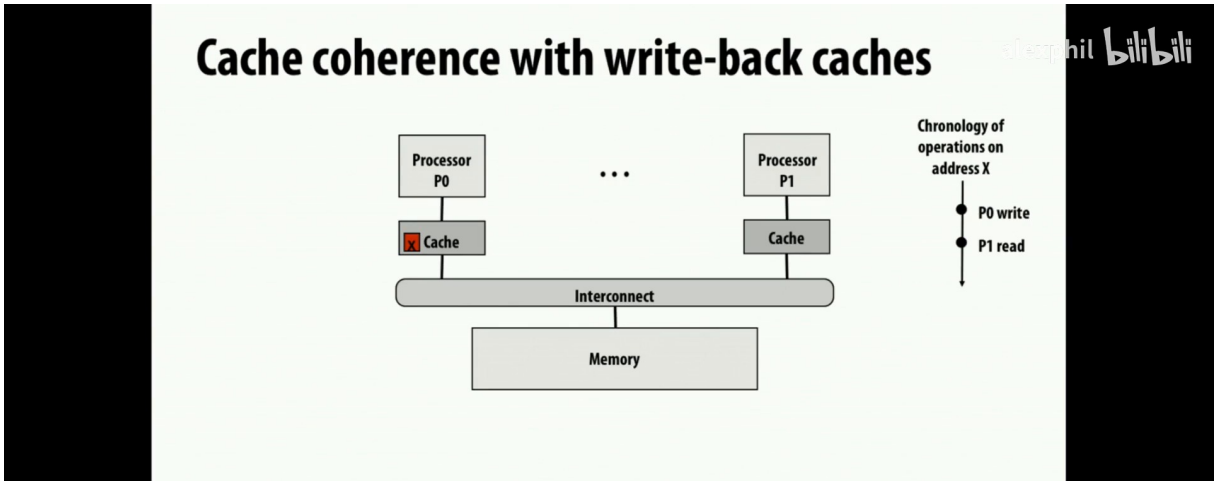


图 21: 回写缓存下的一致性问题：内存可能持有过时的值[21]

讲者说明为什么要专门讨论回写缓存下的一致性：此时内存里的值可能已经过时，最新的副本藏在某个缓存里。协议必须知道最新副本在哪。

[20] 视频画面时间区间：00:36:30–00:36:45。若为裁剪图，时间区间仍对应原视频画面。

[21] 视频画面时间区间：00:37:30–00:37:45。若为裁剪图，时间区间仍对应原视频画面。

6.2 三个状态

MSI 的名字来自它的三个基本状态。讲者的做法是把”有效”拆成两部分。

共享状态：存在可能的多份副本，它们彼此相同。

已修改状态：只存在于某一个缓存中，且可能与内存不同（脏）。

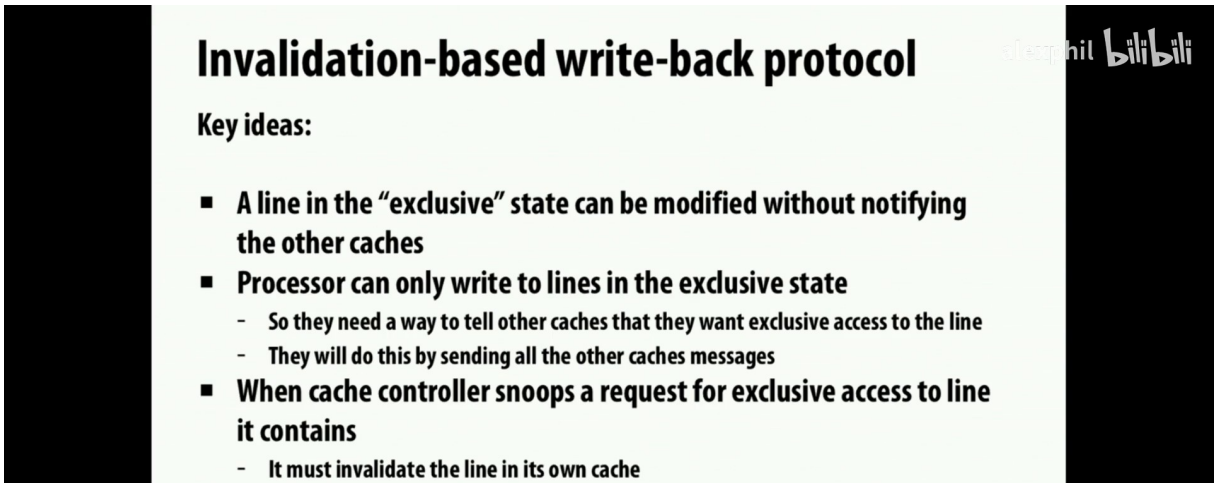


图 22: 基于失效的回写协议：核心要点是独占状态下可本地任意修改[22]

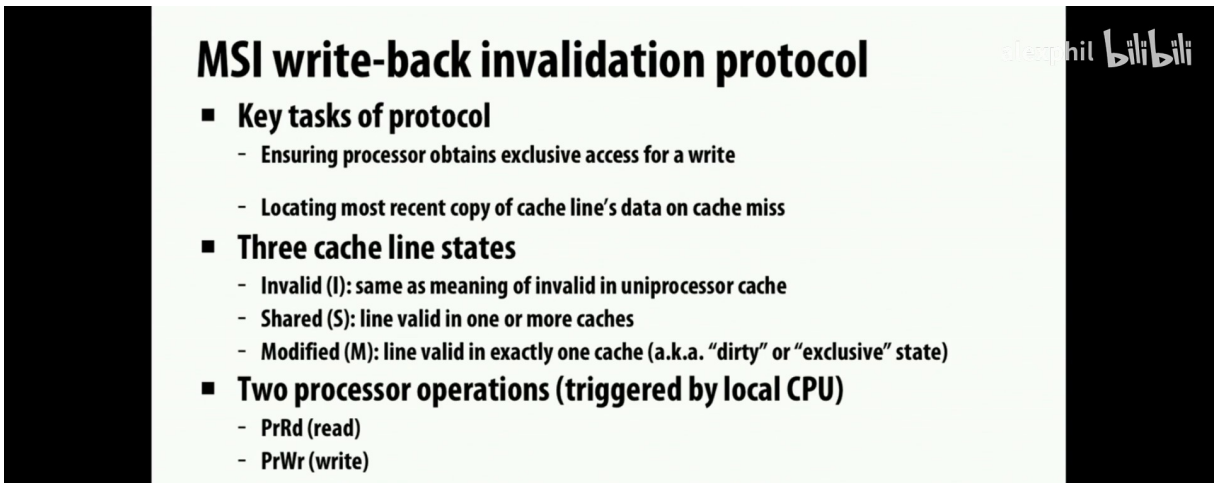


图 23: MSI 回写失效协议：协议需要完成的关键任务[23]

6.3 总线上的三类事务

讲者列出互连上可能出现的三类事务：

只读读入——想要一份副本，但计划只读。

独占读入——想要读入，但计划随后就修改。之所以还需要读，是因为缓存行可能不止一个字，需要保留其余部分。

写回或刷新——把修改过的数据交出去。

[22]视频画面时间区间：00:40:30–00:40:45。若为裁剪图，时间区间仍对应原视频画面。

[23]视频画面时间区间：00:41:30–00:41:45。若为裁剪图，时间区间仍对应原视频画面。

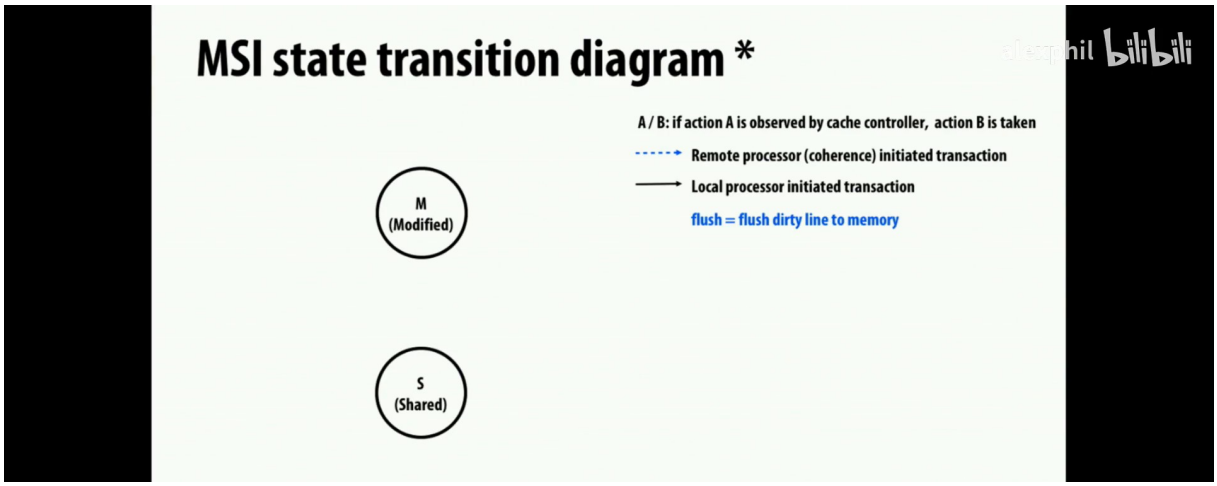


图 24: MSI 状态转移图：标注触发动作与随之执行的动作[24]

6.4 状态转移的关键行为

讲者逐条讲了迁移，其中最有价值的是几个” 为什么”。

处于已修改状态时，处理器可以在本地反复写而无需通知任何人。这正是回写的优势：只要没有其他处理器想要这个值，它就表现得和普通的回写缓存一样。

处于共享状态时，如果听到别的缓存要读这一行，那没关系——只是在更多人之间共享而已。讲者强调这里的” 共享” 不是” 真的在共享”，而是” 可能在共享”：其他处理器后来丢弃副本也不影响。

但若听到某个缓存要做独占读（即有人要写），就必须放弃自己的副本，直接标记为无效，不必再通告什么。

M 状态承担了” 数据唯一持有者” 的责任

处于 M 状态意味着内存是过时的，最新值只存在于这一个缓存。因此任何他人要读或要写，都必须由它负责交出数据。这既是 M 状态的收益来源（本地可自由写），也是它的负担来源（必须随时准备响应他人）。协议设计中” 脏” 与” 独占” 绑定在同一个状态上，正是这个权衡的体现。

若是自己处于已修改状态而别人要读，则必须把数据刷出——从自己的缓存推出去，让对方在数据经过时装载；同时因为别人要写，自己这一份也要作废。

6.5 一个执行示例

讲者用一个两个地址、两个处理器的读写序列把协议走了一遍，展示状态如何随时间演进。

[24] 视频画面时间区间：00:43:30–00:43:45。若为裁剪图，时间区间仍对应原视频画面。

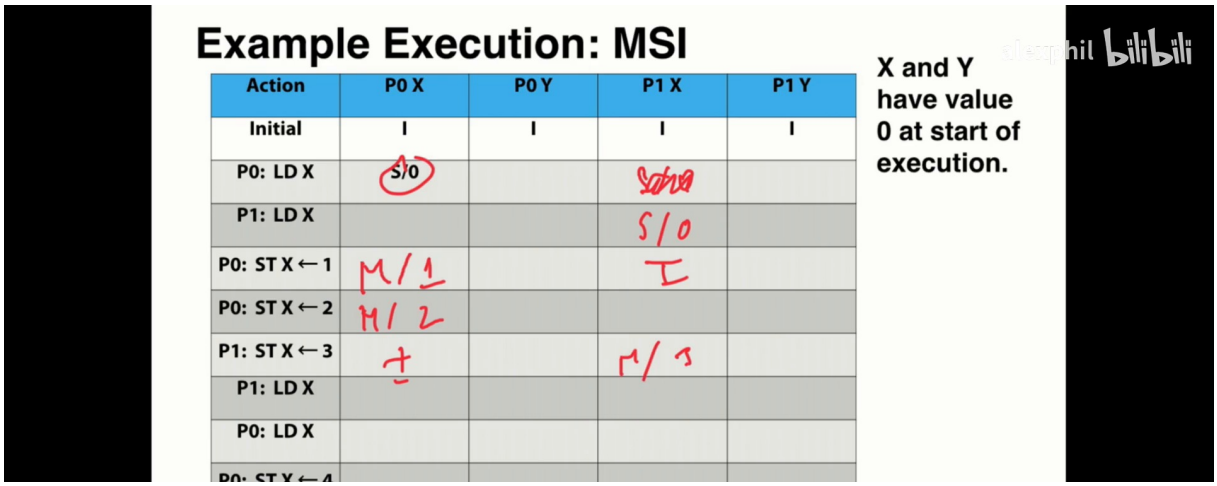


图 26: MSI 执行示例续：状态随操作推进[26]

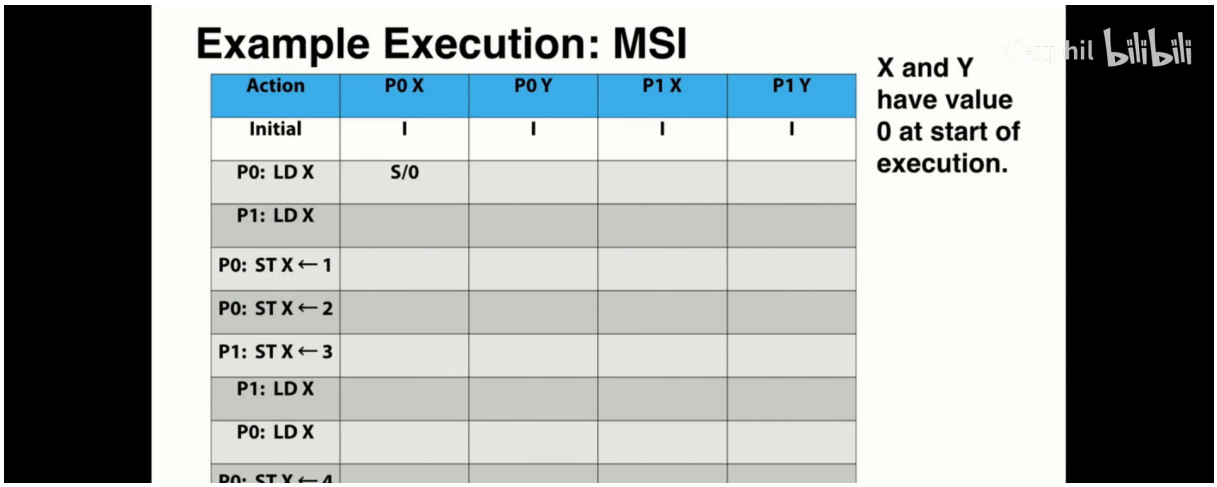


图 25: MSI 执行示例：X 与 Y 的读写序列与状态变化[25]

他的结论是：每个缓存为它持有的每一行维护状态；若某行不在缓存里，隐含就是无效。持有的行必须标记为无效、共享或已修改三者之一。

[25] 视频画面时间区间：00:51:00–00:51:15。若为裁剪图，时间区间仍对应原视频画面。

[26] 视频画面时间区间：00:58:00–00:58:15。若为裁剪图，时间区间仍对应原视频画面。

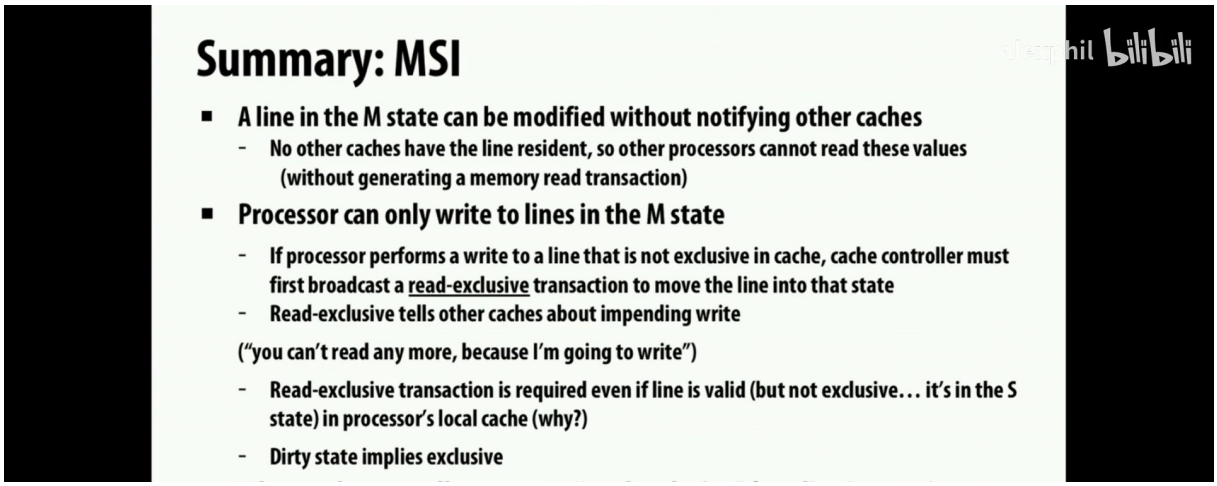


图 27: MSI 小结：M 状态的行可本地修改而无需通知其他缓存[27]

6.6 本章小结

MSI 把状态从两个扩展为三个：无效、共享、已修改。核心收益是已修改状态允许本地反复写而不产生流量，这是回写缓存的优势得以保留的地方。

但代价也清楚了：一旦别人要写，就必须来回交接。

6.7 MSI 是否满足一致性

讲者在这一节点上回过头检验定义。

写传播是通过"失效加刷出"的组合实现的：一方要写就使他人作废，持脏者则把数据交出。写串行化则依靠互连：他假设互连上所有消息按全局顺序发送、也按同一顺序接收，通常由某种总线实现。正是这条互连提供了所需的全序。

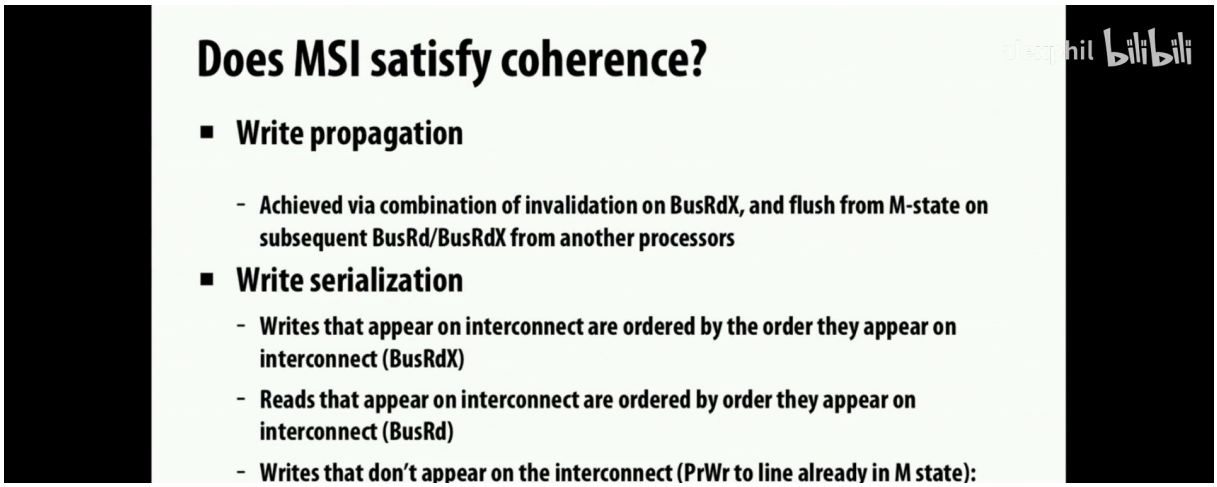


图 28: MSI 是否满足一致性：写传播与写串行化的讨论[28]

[27] 视频画面时间区间：00:58:30–00:58:45。若为裁剪图，时间区间仍对应原视频画面。

[28] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

一致性协议的正确性依赖互连的顺序保证

协议的串行化能力来自互连，而不是来自缓存自身。若互连不保证全局顺序（例如改用某些非一致性的网络拓扑），同一份协议就不再满足写串行化，需要额外机制补上。这解释了为什么 snooping 通常与总线型互连绑定。

7 MSI 的代价与 MESI 的改进

7.1 来回争抢

讲者指出 MSI 一个明显的问题：单核执行时它表现得和普通缓存一样，这很好；但它会导致大量来回争抢。

MSI 的问题在于它无法区分” 独享” 与” 共享”

共享状态只表示” 可能存在多份”，不表示” 确实有多份”。因此即使实际只有自己持有，也必须按最坏情况广播。MESI 的独占状态正是补上这条缺失信息：明确记录” 只有我有，且与内存一致”，从而省掉无谓的广播。

每次想写，都得先升级自己的副本，同时让其他人的副本失效。而且第一次写时，即便别人都没有副本，你也要在总线上说一声我¨ 马上要写这个。¨

于是可以想象加入更多状态，以减少总线流量、避免这类通信开销。

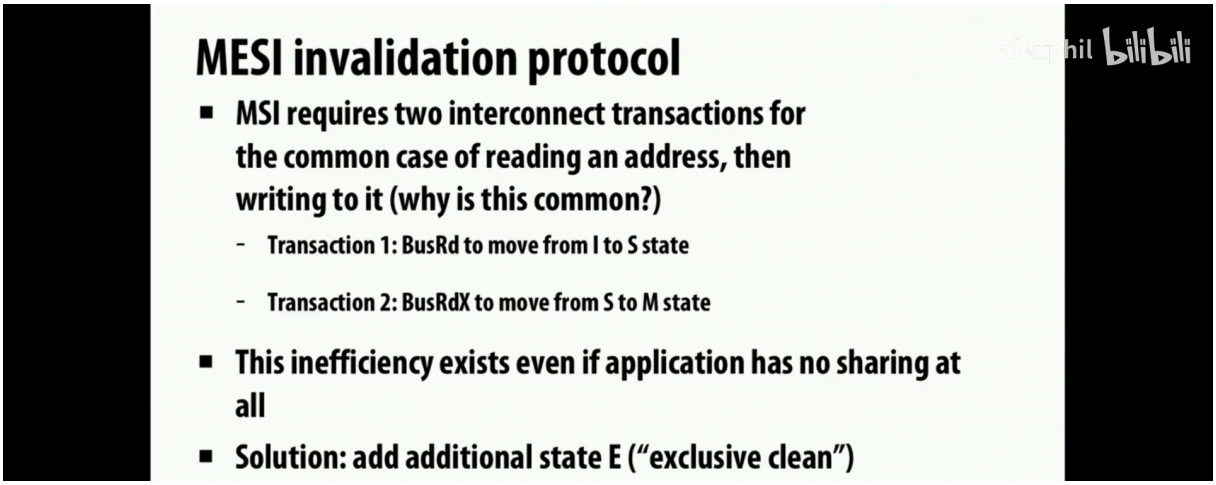


图 29: MESI 失效协议：MSI 在最常见情形下需要两次互连事务[29]

讲者给出的动机很具体：MSI 在最常见的情形下需要两次互连事务。加入独占（Exclusive）状态后，若一行只被一个处理器持有且是干净的，它就可以知道” 没有别人有副本”，从而直接写而不必广播。

事务数可以写成对比：

$$N_{\mathrm{MSI}} = 2, \qquad N_{\mathrm{MESI}} = 1$$

- N_{MSI}：MSI 下独占升级所需的互连事务数，一次独占读加一次后续交互；

[29] 视频画面时间区间：01:02:00–01:02:15。若为裁剪图，时间区间仍对应原视频画面。

- N_{MESI}：有了独占状态后，本已独占比时的情况只需零额外事务；
- 前提是该行无人共享，私有写占主要比例。

7.2 伪共享：程序员的视角

讲者最后转向这个协议的实践意义：你可能遇到大量的来回争抢，而原因往往与逻辑上共享的数据无关。

伪共享的机制是这样的：缓存行是状态记录与失效的单位，而不是变量。两个变量落在同一行里，即使分别只被不同处理器使用，一个处理器的写也会让另一个的副本失效。

讲者描述的场景是两个处理器各自反复写自己那个变量，结果这一行在两个缓存之间来回弹跳，每次都产生互连流量。而这些变量在程序逻辑上毫无关系。

弹跳产生的互连事务数近似为：

$$N_{\text{bounce}} \approx \min(W_1, W_2)$$

- W_1, W_2：两个处理器各自对同一缓存行的写次数；
- 每次交接需要一次失效与一次数据传递，因此事务数与较少的那个写次数同阶；
- 若两者写次数相近，互连流量几乎与总写次数成正比。

伪共享的代价与数据布局直接相关

把每个线程高频写的计数器、标志位放到独立的缓存行上（通常用填充或对齐声明），就能消除这类弹跳。判断依据是”写的目标是否落在同一行”，而不是”逻辑上是否共享”。这是并行代码里最常见、也最容易忽略的一类性能损失。

7.3 不同系统的取舍

讲者最后提到一个对照：GPU 放弃了通用的一致性保证。它的论证是，让软件自己保证不做任何”花哨”的内存操作，就可以提供成本更低、性能更高的实现，因为它不需要那套同步论证。

他说明这正是 GPU（至少 NVIDIA 的 GPU）相对于常规处理器所做的设计取舍。至此本讲结束。

一致性是一份可选的保证

并非所有并行硬件都提供通用一致性。提供它意味着硬件必须在每次访问时维护跨核的顺序与可见性，这份成本由所有代码分担。放弃它则要求软件显式管理，换取更简单的硬件与更高的峰值效率。选择哪种，取决于目标应用是否需要任意的共享内存通信。

8 总结与延伸

这一讲从”缓存制造了单一内存的错觉”出发，走完了一条从问题到协议的完整路径。

缓存行、偏移、回写与写直达、脏位，这些单机机制在多处理器下互相冲突：两个处理器各持副本，一个写入后另一个仍是旧值。问题必须在硬件里解决，因为时间尺度在纳秒级；而在真实机器上，一致性逻辑插在私有缓存与共享缓存之间。要让”解决”有明确含义，需要三条不变量：单处

理器的读写顺序、写的最终可见性、同一地址写的全局顺序一致。实现方案中软件太慢，于是把共享缓存当作监听中心：各控制器在互连上广播事务、互相作废副本。写直达加失效的两状态协议是最小可行版本，但每次写都上总线。MSI 把有效拆成共享与已修改，使持有独占副本的处理器可以本地反复写而无需通告；代价是一旦有他人要写就要来回交接，最常见情形需要两次互连事务。协议的正确性依赖互连提供的全局顺序。最后，伪共享揭示了这套机制的实践代价：状态以行为单位，因此逻辑上无关的变量也会互相拖慢。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 状态数增加是为了省事务，不是为了正确性。两状态、三状态、四状态协议都满足同样的三条不变量，区别只在互连流量。因此评价一个协议，要看它在常见访问模式下产生多少次事务。
- 一致性保证的是” 单一地址的顺序”，不是” 多个地址之间的顺序”。三条不变量只约束同一地址上的操作，跨地址的先后关系属于内存一致性模型的范畴，那是另一讲的主题。
- 伪共享是协议设计的直接后果。只要状态以缓存行为单位，就必然存在这种可能。因此它是程序员需要通过数据布局来规避的问题，而不是协议本身的缺陷。
- 一致性是一份可选的能力。GPU 放弃通用一致性换取效率，说明” 所有并行系统必须一致” 是误解。是否需要，取决于应用是否依赖任意的共享内存通信。

这堂课的边界是它只讲了 snooping，且下一讲才会讨论目录式协议等更具扩展性的方案。讲者反复用” 暂不考虑缓存行多字” 来简化讨论，因此实际协议中” 为改一个字先读整行” 的问题并未展开。

两个可以直接从本讲数据提出的下一步问题：其一，既然 snooping 的流量随核心数增长，那么在多少核之后目录式方案会胜出，判据是什么？其二，MESI 的独占状态只在” 没有其他人有副本” 时才有益，那么当两个线程确实频繁共享同一变量时，独占状态是否完全无益，此时应当改用何种同步方式？